`MarketServiceClient` class. gRPC uses this method to close the channel with the server.

Let's look at one of these methods in the implementation of the service: `GetState` in `MarketServiceImplSharding`. This listing shows how the shard region is created and how the gRPC message is converted to the market protocol in the request and vice versa in the response.

Listing 12.14 Market service implementation

```
class MarketServiceImplSharding(implicit sharding: ClusterSharding)
    extends MarketService {

  implicit val timeout: Timeout = 3.seconds
  implicit val executionContext: ExecutionContext =
    ExecutionContext.global

  val shardingRegion =                                                    ❶
    sharding.init(Entity(Market.typeKey)(entityContext =>
      Market(entityContext.entityId)))

  ...

  override def getState(in: example.market.grpc.MarketId)                 ❷
      : scala.concurrent.Future[example.market.grpc.MarketData] = {
    val market = sharding.entityRefFor(Market.typeKey, in.marketId)       ❸

    market.ask(Market.GetState).mapTo[Market.CurrentState].map {          ❹
      state =>
        val (
          marketId,
          Market.Fixture(id, homeTeam, awayTeam),
          Market.Odds(winHome, winAway, draw)) = (
          state.status.marketId,
          state.status.fixture,
          state.status.odds)

        MarketData(
          marketId,
          Some(FixtureData(id, homeTeam, awayTeam)),
          Some(OddsData(winHome, winAway, draw)))
    }
  }
```